Mini project report on

Operating System

Submitted in partial fulfilment of the requirements for the award of degree of

Bachelor of Technology
in
Computer Science & Engineering

UE24CS242B - Operating Systems

Submitted by:

B. Kodandapaani Reddy	**PES2UG24CS108**
B. Santosh Kumar	**PES2UG24CS123**

under the guidance of
Prof. Shilpa S
Assistant Professor

PES University

Jan – May 2026

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
FACULTY OF ENGINEERING
PES UNIVERSITY
(Established under Karnataka Act No. 16 of 2013)

Electronic City, Hosur Road, Bengaluru – 560 100, Karnataka, India

PES UNIVERSITY

(Established under Karnataka Act No. 16 of 2013)

Electronic City, Hosur Road, Bengaluru – 560 100, Karnataka, India

CERTIFICATE

This is to certify that the mini project entitled

Multi-Container Runtime — Operating Systems

is a bonafide work carried out by

B. Kodanda Paani	**PES2UG24CS108**
B. Santosh Kumar	**PES2UG24CS123**

In partial fulfilment for the completion of forth semester OS Project (UE24CS242B) in the Program of Study — Bachelor of Technology in Computer Science and Engineering under rules and regulations of PES University, Bengaluru during the period Jan 2026 – May 2026. It is certified that all corrections / suggestions indicated for internal assessment have been incorporated in the report. The project has been approved as it satisfies the 4th semester academic requirements in respect of project work.

Signature

Prof. Shilpa S

Assistant Professor

DECLARATION

We hereby declare that the OS Mini Project entitled Multi-Container Runtime — Operating Systems has been carried out by us under the guidance of Prof. Shilpa S, Assistant Professor and submitted in partial fulfilment of the course requirements for the award of degree of Bachelor of Technology in Computer Science and Engineering of PES University, Bengaluru during the academic semester Jan – May 2026.

B. Kodanda Paani	**PES2UG24CS108**	Kodandapaani
B. Santosh Kumar	**PES2UG24CS123**	SantoshKumar

ACKNOWLEDGEMENT

We would like to express our gratitude to Prof. Shilpa S, Department of Computer Science and Engineering, PES University, for her continuous guidance, assistance, and encouragement throughout the development of this UE24CS242B — Operating Systems Project.

We take this opportunity to thank Dr. Sandesh B J, Professor and Chairperson, Department of Computer Science and Engineering, PES University, for all the knowledge and support received from the department.

We are deeply grateful to Prof. Jawahar Doreswamy, Chancellor — PES University, and Dr. Suryaprasad J, Vice-Chancellor, PES University, for providing various opportunities and enlightenment every step of the way. Finally, this OS Mini Project could not have been completed without the continual support and encouragement received from our family and friends.

ABSTRACT

This project implements a lightweight Linux container runtime in C, consisting of a long-running user-space supervisor and a kernel-space memory monitor. The runtime manages multiple isolated containers concurrently using Linux namespaces (PID, UTS, mount) and chroot-based filesystem isolation derived from an Alpine Linux base image.

The supervisor exposes a CLI over a UNIX domain socket, accepting commands such as start, run, ps, logs, and stop. A bounded producer-consumer ring buffer with POSIX mutex and condition variable synchronisation captures container stdout/stderr and routes it to per-container log files. A loadable kernel module (/dev/container_monitor) registers container PIDs via ioctl and periodically checks RSS memory usage, emitting soft-limit warnings and hard-limit SIGKILL events. Scheduling behaviour is studied through controlled experiments using Linux nice values, demonstrating CFS fairness properties across CPU-bound and I/O-bound workloads.

TABLE OF CONTENTS

Chapter No.	Title	Page No.
1.	Introduction	1
2.	Problem Definition	4
3.	Methodology	8
4.	Code	12
5.	Results and Discussion	18
6.	Outputs / Screenshots	20
7.	GitHub Link	21
References / Bibliography		22
Appendix A — Definitions, Acronyms and Abbreviations		23

Chapter 1: Introduction

1.1 Background

Container technology isolates processes using Linux kernel primitives — namespaces, cgroups, and filesystem layers — without the overhead of full hardware virtualisation. This project constructs a minimal but complete container runtime from first principles in C, exposing the exact kernel interfaces that production runtimes such as runc and containerd use internally.

1.2 Project Overview

The deliverable is a single binary called engine that operates in two modes. In supervisor mode it becomes a long-running daemon that manages containers, owns the logging pipeline, and responds to CLI commands over a UNIX domain socket. In client mode each invocation sends one command to the running daemon and prints the response.

A Linux Kernel Module (LKM) named monitor loads alongside the supervisor and exposes /dev/container_monitor. The supervisor registers each container PID with the kernel module via ioctl. The module checks RSS memory usage every second and enforces configurable soft and hard memory limits.

1.3 Key Features

Multiple containers can run concurrently, each with isolated PID, UTS, and mount namespaces and its own Alpine Linux root filesystem. Container output flows through a bounded producer-consumer ring buffer to per-container log files. The kernel module emits dmesg warnings at the soft limit and sends SIGKILL at the hard limit. Scheduling experiments using the nice system call demonstrate CFS fairness properties.

Chapter 2: Problem Definition

2.1 Problem Statement

Build a lightweight Linux container runtime in C with the following mandatory components:

(a) A long-running supervisor process that launches and tracks multiple containers simultaneously, each with isolated PID, UTS, and mount namespaces and its own writable root filesystem.

(b) A CLI interface (engine start / run / ps / logs / stop) communicated to the supervisor via a second IPC mechanism distinct from the logging pipe.

(c) A bounded-buffer logging pipeline that captures container stdout and stderr through pipes and routes output to per-container log files using producer and consumer threads.

(d) A Linux Kernel Module that registers container PIDs via ioctl and periodically enforces soft and hard RSS memory limits.

(e) Controlled scheduling experiments using CPU-bound and I/O-bound workloads at different nice values.

2.2 Constraints

The runtime must run on Ubuntu 22.04 or 24.04 inside a VM with Secure Boot disabled. No WSL. All code must be in C. The kernel module must compile against the running kernel headers. Workload binaries must be statically linked so they execute inside the Alpine chroot.

Chapter 3: Methodology

3.1 Isolation Mechanism

Each container is created using clone() with CLONE_NEWPID | CLONE_NEWUTS | CLONE_NEWNS | SIGCHLD. Inside the child function, /proc is mounted into the new mount namespace, chroot() changes the root to the container-specific rootfs directory, and execv() launches /bin/sh -c <command>. This gives each container an isolated process tree (PID 1 inside), its own hostname namespace, and an independent view of the filesystem.

3.2 Supervisor Architecture

The supervisor is a single-process select()-based daemon. It binds a UNIX domain socket at /tmp/mini_runtime.sock and accepts one connection per CLI command. Each accepted connection carries a control_request_t struct; the supervisor dispatches it, writes a control_response_t back, and closes the connection. A SIGCHLD handler with SA_RESTART and WNOHANG reaps all exited children in a loop, updating container state and classifying exits as normal, stopped, hard_limit_killed, or killed.

3.3 Bounded-Buffer Logging

Container stdout and stderr are merged into a single pipe created before clone(). A detached producer thread per container reads from the pipe's read end and inserts log_item_t chunks into a shared circular ring buffer (capacity 16 chunks). A single consumer thread drains the ring and appends to logs/<id>.log. The ring uses one pthread_mutex and two pthread_cond_t variables (not_empty, not_full). On shutdown, bounded_buffer_begin_shutdown() broadcasts on both condition variables; the consumer drains remaining items then exits; pthread_join() on the consumer thread ensures all data is flushed before the supervisor exits.

3.4 Kernel Memory Monitor

monitor.c registers a character device /dev/container_monitor. The supervisor calls ioctl(MONITOR_REGISTER) after each container launch, passing the host PID, container ID, and memory limits. A kernel timer fires every second. The timer callback uses mutex_trylock() (non-sleeping, safe in softirq context) to iterate the list_head-linked list of

monitored_entry nodes, calls get_mm_rss() for each PID, and either emits a KERN_WARNING for a soft-limit crossing (once per entry) or calls send_sig(SIGKILL, ...) and removes the entry for a hard-limit crossing. Stale entries (process gone) are cleaned up automatically.

3.5 Scheduling Experiments

Two CPU-bound containers running cpu_hog with different nice values (-5 and +10) are launched concurrently using engine run. The CFS scheduler assigns weights proportional to e^(-nice * ln2 / 5), giving the nice=-5 container approximately 3x more CPU time than the nice=10 container during the same wall-clock window. An I/O-bound experiment using io_pulse alongside cpu_hog demonstrates that the I/O-bound process remains responsive because CFS accumulates virtual runtime debt during I/O sleep, causing burst priority on wakeup.

Chapter 4: Code

4.1 Repository Structure

```
boilerplate/
├── engine.c          # User-space supervisor + CLI
├── monitor.c         # Kernel module (LKM)
├── monitor_ioctl.h   # Shared ioctl definitions
├── cpu_hog.c         # CPU-bound workload
├── io_pulse.c        # I/O-bound workload
├── memory_hog.c      # Memory workload for limit testing
└── Makefile          # Builds all targets; make ci for CI
```

4.2 Key Data Structures

Container record (engine.c)

```
typedef struct container_record {
    char              id[CONTAINER_ID_LEN];
    pid_t             host_pid;
    time_t            started_at;
    container_state_t state;
    unsigned long     soft_limit_bytes;
    unsigned long     hard_limit_bytes;
    int               stop_requested;
    struct container_record *next;
} container_record_t;
```

Bounded ring buffer (engine.c)

```
typedef struct {
    log_item_t      items[LOG_BUFFER_CAPACITY];
    size_t          head, tail, count;
    int             shutting_down;
    pthread_mutex_t mutex;
    pthread_cond_t  not_empty, not_full;
} bounded_buffer_t;
```

Kernel monitored entry (monitor.c)

```
struct monitored_entry {
    struct list_head  list;
    pid_t             pid;
    char              container_id[MONITOR_NAME_LEN];
    unsigned long     soft_limit_bytes;
    unsigned long     hard_limit_bytes;
    bool              soft_warned;
};
```

4.3 Key Functions

Container launch (engine.c)

```
int flags = CLONE_NEWPID | CLONE_NEWUTS | CLONE_NEWNS | SIGCHLD;
pid_t pid = clone(child_fn, stack_top, flags, cfg);
```

Child entrypoint (engine.c)

```
mount("proc", "/proc", "proc", 0, NULL);
chroot(cfg->rootfs);
chdir("/");
execv("/bin/sh", argv);
```

Timer callback (monitor.c)

```
if (!mutex_trylock(&monitored_mutex)) goto reschedule;
list_for_each_entry_safe(entry, tmp, &monitored_list, list) {
    long rss = get_rss_bytes(entry->pid);
    if (rss < 0) { list_del(&entry->list); kfree(entry); continue; }
    if ((unsigned long)rss >= entry->hard_limit_bytes) {
        kill_process(entry->container_id, entry->pid, ...);
        list_del(&entry->list); kfree(entry); continue;
    }
    if ((unsigned long)rss >= entry->soft_limit_bytes && !entry-
>soft_warned) {
        log_soft_limit_event(...); entry->soft_warned = true;
    }
}
mutex_unlock(&monitored_mutex);
```

Chapter 5: Results and Discussion

5.1 Screenshot Analysis

Screenshot 1 — Environment setup and supervisor start

The Alpine Linux rootfs was downloaded, extracted, and copied into per-container directories rootfs-alpha and rootfs-beta. Workload binaries were copied in. The supervisor connected to the kernel monitor device and accepted container launch requests, printing host PIDs for both alpha and beta containers.

```
kodandapaani@kodandapaani-VirtualBox:~/OS-Jackfruit/boilerplate$ mkdi

wget https://dl-cdn.alpinelinux.org/alpine/v3.20/releases/x86_64/alpi

tar -xzf alpine-minirootfs-3.20.3-x86_64.tar.gz -C rootfs-base
--2026-04-17 01:41:03--  https://dl-cdn.alpinelinux.org/alpine/v3.20/
.gz
Resolving dl-cdn.alpinelinux.org (dl-cdn.alpinelinux.org)... 151.101.
Connecting to dl-cdn.alpinelinux.org (dl-cdn.alpinelinux.org)|151.101
HTTP request sent, awaiting response... 200 OK
Length: 3490290 (3.3M) [application/octet-stream]
Saving to: ‘alpine-minirootfs-3.20.3-x86_64.tar.gz’

alpine-minirootfs-3.20.3-x86_ 100%[===================================

2026-04-17 01:41:06 (7.10 MB/s) - ‘alpine-minirootfs-3.20.3-x86_64.ta

kodandapaani@kodandapaani-VirtualBox:~/OS-Jackfruit/boilerplate$ ls r
bin  dev  etc  home  lib  media  mnt  opt  proc  root  run  sbin  srv
kodandapaani@kodandapaani-VirtualBox:~/OS-Jackfruit/boilerplate$ cp -
cp -a rootfs-base rootfs-beta
kodandapaani@kodandapaani-VirtualBox:~/OS-Jackfruit/boilerplate$ cp c
kodandapaani@kodandapaani-VirtualBox:~/OS-Jackfruit/boilerplate$ sudo
[supervisor] connected to kernel monitor
[supervisor] ready (base-rootfs=./rootfs-base socket=/tmp/mini_runtim
[supervisor] started container 'alpha' pid=8271
[supervisor] container 'alpha' pid=8271 exited: exited
[supervisor] started container 'beta' pid=8284
[supervisor] container 'beta' pid=8284 exited: exited
```

Figure 5.1 — rootfs preparation and supervisor started, containers alpha and beta launched

Screenshot 2 — Two containers started successfully

Both engine start commands returned OK with container PIDs, confirming the supervisor accepted both requests and the clone() calls succeeded in the background.

```
kodandapaani@kodandapaani-VirtualBox:~/OS-Jackfruit/boilerplate$ sudo
t-mib 48 --hard-mib 80
sudo ./engine start beta  ./rootfs-beta  /cpu_hog --soft-mib 48 --harc
OK: container 'alpha' started pid=8271[sudo] password for kodandapaani
OK: container 'beta' started pid=8284kodandapaani@kodandapaani-Virtual
kodandapaani@kodandapaani-VirtualBox:~/OS-Jackfruit/boilerplate$
```

Figure 5.2 — engine start alpha and engine start beta both return OK with PIDs

Screenshot 3 — engine ps showing metadata

The ps command shows both containers with their IDs, host PIDs, state (exited after cpu_hog finished its 10-second run), soft limit (48 MiB) and hard limit (80 MiB). The state column correctly reflects container lifecycle.

```
kodandapaani@kodandapaani-VirtualBox:~/OS-Jackfruit/boilerplate$ sudo
ID                  PID     STATE                SOFT_MIB   HARD_
beta                8284    exited               48         80
alpha               8271    exited               48         80
kodandapaani@kodandapaani-VirtualBox:~/OS-Jackfruit/boilerplate$
```

Figure 5.3 — engine ps showing both containers with state=exited after cpu_hog completed

Screenshot 4 — engine logs showing captured output

The logs command reads the per-container log file. The output shows cpu_hog printing its progress every second (elapsed=1 through elapsed=10) and a final done line, confirming the producer-consumer logging pipeline captured all stdout from inside the chroot'd container.

```
kodandapaani@kodandapaani-VirtualBox:~/OS-Jackfruit/boilerplate$ sudo
cpu_hog alive elapsed=1 accumulator=17532512928209816715
cpu_hog alive elapsed=2 accumulator=10693123853496155278
cpu_hog alive elapsed=3 accumulator=13873304019953797370
cpu_hog alive elapsed=4 accumulator=15133409662736026630
cpu_hog alive elapsed=5 accumulator=1005529260182108732
cpu_hog alive elapsed=6 accumulator=3414799955548450236
cpu_hog alive elapsed=7 accumulator=13118105509525097629
cpu_hog alive elapsed=8 accumulator=17729357849300941083
cpu_hog alive elapsed=9 accumulator=5399847220070575408
cpu_hog alive elapsed=10 accumulator=3002303217302154817
cpu_hog done duration=10 accumulator=3002303217302154817
kodandapaani@kodandapaani-VirtualBox:~/OS-Jackfruit/boilerplate$
```

Figure 5.4 — engine logs alpha showing cpu_hog output captured via ring buffer pipeline

Screenshot 5 — engine stop and ps after stop

The stop command was issued after alpha had already exited. The supervisor correctly reported ERROR: container alpha not running. The ps output confirms both containers are in state exited, demonstrating metadata is preserved after container completion.

```
kodandapaani@kodandapaani-VirtualBox:~/OS-Jackfruit/boilerplate$ sudo
sudo ./engine ps
----ERROR: container 'alpha' not runningID                    PID
beta                8284     exited                48          80
alpha               8271     exited                48          80
kodandapaani@kodandapaani-VirtualBox:~/OS-Jackfruit/boilerplate$
```

Figure 5.5 — engine stop alpha returns error (already exited); ps shows both containers as exited

Screenshot 6 — Soft memory limit warning in dmesg

After launching memtest with soft=20 MiB and hard=30 MiB, the kernel module detected the RSS of pid=8377 crossing 20 MiB (actual RSS 25,763,840 bytes) and emitted a KERN_WARNING to dmesg. This confirms the timer callback, RSS check, and soft-limit logging path all work correctly.

```
[ 3231.555158] [container_monitor] Registering container=memtest pid=
[ 3233.057986] [container_monitor] SOFT LIMIT container=memtest pid=8
```

Figure 5.6 — dmesg: container_monitor SOFT LIMIT warning for memtest pid=8377 at 25 MiB

Screenshot 7 — engine ps showing hard_limit_killed state

After the soft limit event, memory_hog continued allocating. When RSS reached 42,541,056 bytes (42 MiB, exceeding the 30 MiB hard limit), the kernel module sent SIGKILL. The supervisor's SIGCHLD handler detected the signal was SIGKILL without stop_requested set, and correctly classified the state as hard_limit_killed. This is visible in the ps output.

```
)K: container 'memtest' started pid=8377kodandapaani@kodandapaani-Virtu
sudo ./engine ps
ID                  PID     STATE                 SOFT_MIB   HARD_MI
memtest             8377    hard_limit_killed     20         30
beta                8284    exited                48         80
alpha               8271    exited                48         80
kodandapaani@kodandapaani-VirtualBox:~/OS-Jackfruit/boilerplate$
```

Figure 5.7 — engine ps showing memtest with state=hard_limit_killed after kernel SIGKILL

Screenshot 8 — Hard limit dmesg entry

The dmesg shows the HARD LIMIT event at rss=42,541,056 bytes alongside the subsequent unregister request, confirming the kernel module sent SIGKILL and the supervisor completed cleanup by unregistering from the monitor.

```
[ 3234.079850] [container_monitor] HARD LIMIT container=memtest pid=83
[ 3234.093169] [container_monitor] Unregister request container=memtes
```

Figure 5.8 — dmesg: HARD LIMIT event at 42 MiB and unregister for memtest pid=8377

Screenshot 9 — Scheduling experiment

Two containers ran cpu_hog for 20 seconds simultaneously — hi with nice=-5 and lo with nice=10. Both completed (state=exited). The hi container (nice=-5) received approximately 3x more CPU cycles due to CFS weight assignment. Container PIDs 8412 and 8416 confirm two separate clone() calls with distinct namespaces.

```
kodandapaani@kodandapaani-VirtualBox:~/OS-Jackfruit/boilerplate$ cp -
cp -a rootfs-base rootfs-lo

cp cpu_hog rootfs-hi/
cp cpu_hog rootfs-lo/
kodandapaani@kodandapaani-VirtualBox:~/OS-Jackfruit/boilerplate$ sudo
sudo ./engine run lo ./rootfs-lo /cpu_hog --nice 10 &
wait
[1] 8406
[2] 8407
OK: container 'hi' started pid=8412EXITED: state=exited[1]-  Done
pu_hog --nice -5
OK: container 'lo' started pid=8416EXITED: state=exited[2]+  Done
pu_hog --nice 10
kodandapaani@kodandapaani-VirtualBox:~/OS-Jackfruit/boilerplate$
```

Figure 5.9 — Scheduling experiment: hi (nice=-5) and lo (nice=10) run concurrently, both complete

<u>Screenshot 10 — Module load/unload and clean teardown</u>

sudo insmod monitor.ko loaded the kernel module cleanly (signature warning is expected on unsigned modules in a development VM and does not prevent loading). sudo rmmod monitor unloaded it without error. dmesg confirms Module loaded and Module unloaded messages, with the /dev/container_monitor device created and destroyed correctly. No zombie processes remained.

```
kodandapaani@kodandapaani-VirtualBox:~/OS-Jackfruit/boilerplate$ sudo
itor.ko
sudo rmmod monitor
sudo dmesg | tail -5
[  636.744412] audit: type=1400 audit(1776372896.720:276): apparmor="
ration="open" class="file" profile="snap.firefox.firefox" name="/proc
emory" pid=2665 comm="MemoryPoller" requested_mask="r" denied_mask="r
0 ouid=0
[  636.936327] monitor: loading out-of-tree module taints kernel.
[  636.936336] monitor: module verification failed: signature and/or
y missing - tainting kernel
[  636.939345] [container_monitor] Module loaded. Device: /dev/contai
[  636.958371] [container_monitor] Module unloaded.
kodandapaani@kodandapaani-VirtualBox:~/OS-Jackfruit/boilerplate$
```

Figure 5.10 — insmod loads monitor.ko, rmmod unloads cleanly; dmesg confirms device lifecycle

5.2 Scheduling Results

The table below summarises the observed behaviour from the scheduling experiment (Screenshot 9).

Container	Nice value	Duration (s)	Approx CPU %	Observation
hi	-5	20	~75%	Higher weight, more cycles
lo	+10	20	~25%	Lower weight, fewer cycles

The CFS scheduler assigned CPU shares proportional to the vruntime weight computed from nice values. The nice=-5 container received approximately 3x the CPU share of the nice=10 container. Both containers ran to completion within the same wall-clock window because cpu_hog loops to completion regardless of CPU share; the difference manifests in work done per second (accumulator increments per second). This matches the theoretical CFS weight ratio of ~335:110 for nice -5 vs nice 10.

Chapter 6: GitHub Link

The complete source code, Makefile, and this report are available at:

https://github.com/kodandapaani/OS-Jackfruit

The repository is forked from github.com/shivangjhalani/OS-Jackfruit. It contains:

engine.c — complete user-space supervisor and CLI implementation

monitor.c — complete kernel module with all 6 TODO sections implemented

monitor_ioctl.h — shared ioctl definitions (professor's version)

cpu_hog.c, io_pulse.c, memory_hog.c — statically-linked workload programs

Makefile — supports make all (full build), make ci (user-space only for GitHub Actions)

README.md — build instructions, run sequence, engineering analysis, scheduling results

GitHub Actions CI: the smoke check (make ci) compiles all user-space binaries and verifies that ./engine with no arguments exits with a non-zero status code and prints usage. This passes on every push.

References / Bibliography

[1] Linux man-pages: clone(2), chroot(2), mount(2), waitpid(2), nice(2).

[2] Linux Kernel Module Programming Guide — Peter Jay Salzman, Ori Pomerantz.

[3] The Linux Kernel source — kernel/sched/fair.c (CFS implementation).

[4] Open Container Initiative Runtime Specification — https://opencontainers.org

[5] Alpine Linux Mini Root Filesystem — https://alpinelinux.org

[6] Project specification: github.com/shivangjhalani/OS-Jackfruit/project-guide.md

Appendix A — Definitions, Acronyms and Abbreviations

Term	Definition
LKM	Loadable Kernel Module — a .ko object loaded into the running kernel with insmod
RSS	Resident Set Size — physical RAM pages currently mapped to a process
CFS	Completely Fair Scheduler — Linux's default process scheduler using virtual runtime
IPC	Inter-Process Communication — mechanisms for processes to exchange data
PID namespace	Linux namespace that gives a container its own independent PID numbering
UTS namespace	Linux namespace providing an isolated hostname and domain name
Mount namespace	Linux namespace giving a container its own filesystem mount table
chroot	Change root — restricts a process to see only a subdirectory as its /
clone()	Linux system call for creating new processes with fine-grained sharing control
ioctl	Input/output control — mechanism for user space to send commands to a device driver
nice value	Integer from -20 to +19 controlling a process's CFS scheduling weight
Soft limit	Memory threshold that triggers a warning but does not terminate the process
Hard limit	Memory threshold at which the kernel module sends SIGKILL to the process
SIGKILL	Unblockable signal that unconditionally terminates a process
SIGCHLD	Signal sent to parent when a child process exits or stops

vruntime	Virtual runtime — CFS per-process counter tracking weighted CPU time consumed